TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

UÇAN ARABA YARIŞMASI

FİNAL TASARIM RAPORU

TAKIM ADI: AKÜ_RISECAR

TAKIM ÜYELERİ: Keziban Karagümüş[1], İrem Çırpan[2],

Berat Mehmet Bakı[3], Enes Güleç[4], Yiğit Yazıcı[5]

DANIŞMAN ADI: Dr. Naim Karasekreter

İçindekiler

1. Hazırlanan senaryolarda kullanılan sensörler ve programlama dili ile ilgili aşağıdaki soruları cevaplayınız. 3

2. Senaryo 1'de, hava aracının B noktasına gidişi için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız. 3

3. Senaryo 4'te, yasak bölgeden kaçış için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız. 5

4. Senaryo 7'de, trafikte olan diğer uçan arabaları gözeterek kaza yapmadan uçmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 12

5. Senaryo 8'de, en yakın hastaneyi bulmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 19

6. Senaryo 10 içerisinde GNSS arızası yaşandığında inişi tamamlayan algoritma için aşağıdaki soruları cevaplayınız. 20

7. Senaryo 11 içerisinde GNSS arızası yaşandığında seyir yapmaya devam eden ve görevi tamamlamayı sağlayan algoritma için aşağıdaki soruları cevaplayınız. 20

8. Senaryo 12'de GNSS arıza durumunu tespit eden algoritma için aşağıdaki soruları cevaplayınız. 21

9. Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimini yaptığınız algoritma için aşağıdaki soruları cevaplayınız. 22

10. Senaryo 15'te manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe hava aracının yönelmesini sağlayan algoritma için aşağıdaki soruları cevaplayınız. 23

11. Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarını tespit ettiğiniz ve hava aracını oraya yönlendirdiğiniz algoritma için aşağıdaki soruları cevaplayınız. 26

1. **Hazırlanan senaryolarda kullanılan sensörler ve programlama dili ile ilgili aşağıdaki soruları cevaplayınız.**
   I. Kod yazarken kullandığınız dil ve özelliklerini açıklayınız.
   II. Bu dil yerine başka hangi programlama dilini tercih ederdiniz? Nedeni ile açıklayınız.
   III. Barometre sensörünün nasıl çalıştığını açıklayınız. GNSS ile barometre irtifa verilerinin farkı nedir?
   IV. İvmeölçer ve dönüölçer sensörlerinde genel hata karakteristikleri nelerdir? Bu karakteristiklerin özelliklerini açıklayınız.

CevapI: Kullanılan dil Python dilidir. Python dili, makine mantığına uzak ancak insan mantığına oldukça yakın olduğundan yüksek seviye olarak kabul edilmektedir. Üst düzey söz dizilimi yanında interaktif ve nesne odaklı bir dil olması sayesinde modül ve yüksek veriler ile birlikte çalışabilmektedir. Masaüstü uygulamalarına ek olarak veri analizli ve web tabanlı uygulamaları da python dili ile programlamak mümkündür.

CevapII: Nesne Yönelimli Programlama (NYP), mantıksal işlemlerden ziyade, nesnelere (object) ve nesneler üzerinde işlemlere odaklanan programlama dili modelidir. Python programlama dilinin nesne tabanlı bir programlama dili olması nedeniyle projede bu dil yerine C++ veya C# gibi başka bir nesne tabanlı bir programlama dili kullanılabilir.

CevapIII: Barometre sensörü hava basıncını temel alır. gnss uydu verisidir.

CevapIV: İvmeölçer: “X-Temp Comp”, “Y-Temp Comp”, “Z-Temp Comp” blokları sıcaklığa bağlı olarak yarıiletken yapıda meydana gelen değişikliklerden dolayı ölçüm hataları vardır. Dönüölçer: Sabit kayma hatası, Orantı katsayısı hatası, Eksenel kaçıklık hatası.

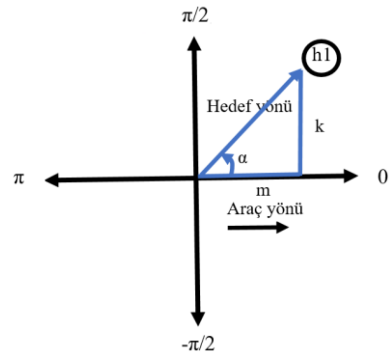
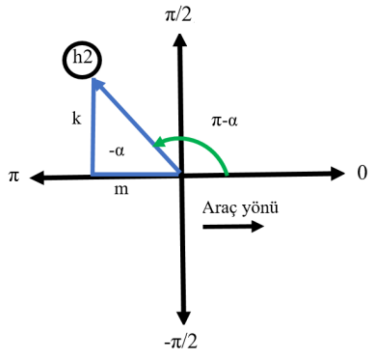
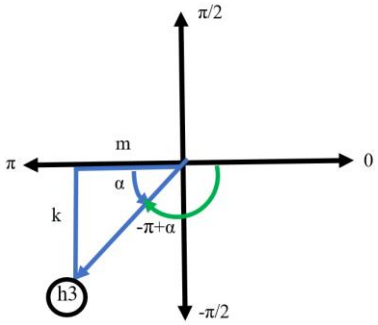
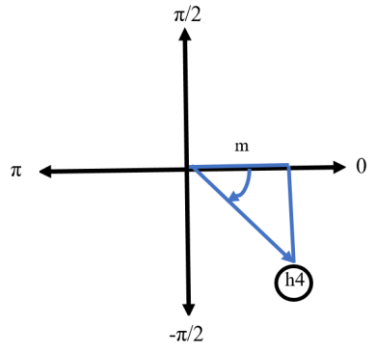
2. **Senaryo 1’de, hava aracının B noktasına gidişi için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız.**
   I. Hava aracının gitmesi gereken yönü hesapladığınız algoritmayı EK-1’de verilen örnekteki gibi açıklayınız.
   II. Hava aracının gitmesi gereken koordinatları hangi veri yapısından öğrendiniz?
   III. Hava aracının inişe geçmesi için gerekli koşul seçimleriniz nelerdir?
   IV. İnişte hangi sensörü irtifa tespiti için kullandınız?
   V. İnişte yere değme hızınızı kaç m/s olarak ayarladınız? Nasıl?

CevapI:

Algoritma	Açıklama
if(hedefevarildimi== False): boylamfark=cezeri.gnss.boylam-varis_bolgesi.boylam enlemfark=cezeri.gnss.enlemvaris_bolgesi.enlem tan=boylamfark/enlemfark aci=math.atan(tan)	Tüm kod satırları while döngün bloğunun içinde çalıştığı için ilk olarak hedefe varılıp varılmadığı kontrol edilir. Eğer varılmadıysa gideceği hedefe doğru aracın dönmesi için dönüş açısı hesaplanır. Açının doğru bir şekilde hesaplanabilmesi için tüm harita iki boyutlu bir düzlem olarak ele alınmıştır ve koordinat sistemi üzerinde 4 bölgeye ayrılmıştır. Hedef, bu dört bölgeden birinde olacağından dönüş açısıda ona göre hesaplanmıştır. Muhtemel hedeflere göre yön tayini için geliştirilen algoritmanın matematiksel açılımı Şekil 1'de verilmiştir.

if(aci-cezeri.manyetometre.veri>0):
if(cezeri.gnss.enlem<varis_bolgesi.enlem): **#1.bolge**
cezeri.don(aci-cezeri.manyetometre.veri)
else:
cezeri.don((aci-cezeri.manyetometre.veri-math.pi))**#3.bolge**
	Bu kısımda, aracın ve hedefin enlem-boylam verilerine bakılarak koordinat düzleminde hedefin konumu belirlenir ve aracın hedefe doğru dönmesi sağlanır. Bu kod bloğu, 1. ve 3. bölgeleri kontrol eder.

else: if(cezeri.gnss.enlem<varis_bolgesi.enlem): **#2.bolge** cezeri.don(aci-cezeri.manyetometre.veri) else: cezeri.don((aci-cezeri.manyetometre.veri+math.pi))**#4.bolge**	Bu kod bloğu, 2. ve 4. bölgeleri kontrol eder.

<table>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
<tr>
<td>$\alpha_{rad} = dönüş\ açısı = arctan$ (k/m)</td>
<td>dönüş açısı=π-α</td>
</tr>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
<tr>
<td>$\alpha_{rad} = arctan$ (k/m)
dönüş açısı = -π+α</td>
<td>$\alpha_{rad} = dönüş\ açısı = arctan$ (k/m)
dönüş açısı= -α</td>
</tr>
</table>

Şekil 1. Hedeflere göre aracın yön tayini algoritması

CevapII: Hedefler, *cezeri.hedefler* dizisinden alınmaktadır. *cezeri.hedefler* dizisinin her indeksi bir bölge belirtir ve bu bölgeyi *varıs_bolgesi* değişkenimize atayıp enlem ve boylam değerleri *varıs_bolgesi.enle*m, *varıs_bolgesi.boylam* değişkenlerine alınmaktadır.

CevapIII: Aracın enlem ve boylam değerlerinin, hedefin enlem ve boylam değerleri ile eşit olması durumunda hedefe varıldığı anlaşılmaktadır.

CevapIV: İrtifa tespiti için radar sensörünü kullanıldı. Radar sensörü hava koşullarından etkilenmeyen bir yükseklik sensörü olduğu için tercih edildi.

CevapV: Yere değme hızı, 4,4 m/sn (*cezeri.yava*s) olarak ayarlandı. *cezeri.radar.mesafe* >16 && *cezeri.radar.mesafe* = 0 ise *cezeri.asagi_git* (hızlı), *cezeri.radar.mesafe* < 16 ise aracın hızı yavaş olarak ayarlandı.

3. Senaryo 4'te, yasak bölgeden kaçış için tasarladığınız algoritma için aşağıdaki soruları cevaplayınız.

I. Yasak bölgeleri tespit ettiğiniz veri yapısı nedir?

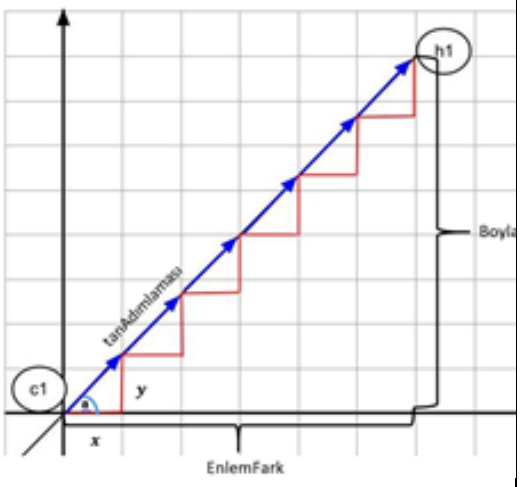
II. Bu bölgelere girmemek için tasarladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: *bolge.ucusa_yasakli_bolge* değişkeni boolean değer alan bir değişkendir. True döndürdüğünde bölgenin yasaklı olduğu anlaşılmaktadır.

CevapII:

Algoritma	Açıklama

yasaklibolgedondurbool=True
def yasakli_bolge_dondur():
global yasaklibolgedondurbool
if yasaklibolgedondurbool:
bolge = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
hedefenlem=varis_bolgesi.enlem
hedefboylam=varis_bolgesi.boylam
kaynakboylam=cezeri.gnss.boylam
kaynakenlem=cezeri.gnss.enlem
tandegisken=tan1(kaynakboylam,kaynakenlem,hedefboylam,hedefenlem)
	Araçla hedef arasında belirlenen rota, ***"tan1"*** fonksiyonu kullanılarak küçük bölgelere ayrılır ve her bölgede yasaklı bölge kontrolü yapılarak ilerlenir.
	Algoritmanın bu kısmında, hedef ve aracın enlem-boylam değerleri farklı değişkenlere atanır ve tanjant hesaplamaları için *tan1* fonksiyonu çağırılır.

if(-1<tandegisken and tandegisken<1):
if(tandegisken>0):
if(hedefboylam>kaynakboylam):
while
 	
(hedefboylam>tandegisken+kaynakboylam):
if bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
yasaklimi=False
kaynakboylam=tandegisken+kaynakboylam
 	
kaynakenlem=kaynakenlem+1
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
else:
	Kodun bu kısmında, rotanın küçük bölgelere ayrılması sağlanır ve her bölge yeni bir hedef olarak Kabul edilir. Rotanın bölgelere ayrılması ile ilgili şekilsel gösterim Şekil 2'de verilmektedir.
	 
	
	Şekil 2. Rotanın bölgelere ayrılması
	Şekil 2'de de görüldüğü gibi, ana rotamız küçük bölgelere ayrılmaktadır. Her bölge bizim için yeni bir hedefi temsil etmektedir.

while (hedefboylam<kaynakboylam-tandegisken):
if bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
yasaklimi=False
kaynakboylam=kaynakboylam-tandegisken
kaynakenlem=kaynakenlem-1
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
if(tandegisken<0):
 	
if(kaynakboylam<hedefboylam):#zeceri sol tarafta
while (hedefboylam>kaynakboylam-tandegisken):
if bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
yasaklimi=False
 	
kaynakboylam=kaynakboylam-tandegisken
 	
kaynakenlem=kaynakenlem-1
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
else:
while (hedefboylam<kaynakboylam+tandegisken):
	En son bölge ise asıl hedef olarak Kabul edilmiştir. Her yeni hedefin enlem ve boylam bilgileri Eşitlik 1 ve Eşitlik 2'de gösterildiği şekilde hesaplanmaktadır.
	Burada, Cezeri(c1) ve Hedef(h1) için enlem/boylam arasındaki farklar alınarak tanjant kuralı uygulanarak aracın rotasında kalması için dönüş açısını temsil eden tanx değeri hesaplanır. Elde edilen tanx değerinin alacağı değere göre rota adımlara ayrılır ve her adım ayrı ayrı geçilerek yasak bölge kontrolleri yapılır. Geçilen adımda, yasak bölge bulunması durumunda yasak bölgenin konumu bir dizide kayıt altına alınır ve bir nevi rotamız üzerinde bulunan yasak bölge koordinatlarının haritası çıkarılarak o bölgelere dokunmadan aracın hareketi sağlanır.

```
                if
bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,ka
ynakenlem])
                    yasaklimi=False
kaynakboylam=kaynakboylam+tandegiske
n
kaynakenlem=kaynakenlem+1
bolge=cezeri.harita.bolge(kaynakenlem,ka
ynakboylam)
elif(tandegisken>1):
        if(hedefboylam>kaynakboylam):
            while
(hedefboylam>kaynakboylam+1):
                if  bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,ka
ynakenlem])
                    yasaklimi=False

kaynakboylam=kaynakboylam+1
kaynakenlem=(1/tandegisken)+kaynakenle
m
bolge=cezeri.harita.bolge(kaynakenlem,ka
ynakboylam)
else:
            while
(hedefboylam<kaynakboylam-1):
                if  bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,ka
ynakenlem])
                    yasaklimi=False

kaynakboylam=kaynakboylam-1
kaynakenlem=kaynakenlem-
(1/tandegisken)
```

```
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
elif(tandegisken<-1):
        if(hedefboylam>kaynakboylam):
            while (hedefboylam>kaynakboylam+1):

kaynakboylam=kaynakboylam+1

kaynakenlem=kaynakenlem+(1/(tandegisken))
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
                if  bolge.ucusa_yasakli_bolge:
                    print('yasakli bolge',kaynakboylam,kaynakenlem)
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
                    yasaklimi=False
else:
            while (hedefboylam<kaynakboylam-1):

kaynakboylam=kaynakboylam-1
                kaynakenlem=kaynakenlem-(1/tandegisken)
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
                if  bolge.ucusa_yasakli_bolge:
                    print('yasakli bolge',kaynakboylam,kaynakenlem)
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
                    yasaklimi=False
else:
```

while (hedefboylam<kaynakboylam-1):
 	
kaynakboylam=kaynakboylam-1
kaynakenlem=kaynakenlem-(1/tandegisken)
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
if bolge.ucusa_yasakli_bolge:
yasaklibolgeler.append([kaynakboylam,kaynakenlem])
yasaklimi=False	

$$\tan x = BoylamFark/EnlemFark \quad (1)$$

$$f(x) = \begin{cases} \frac{y=kaynakboylam-tanx}{x=kaynakenlem-1} & -1 < tanx < 0 \ \ and\ hedefboylam > kaynakboylam \\ \frac{y=kaynakboylam-tandegisken}{x=kaynakenlem-1} & 1 < tanx < 0 \ \ and\ hedefboylam < kaynakboylam \\ \frac{y=kaynakboylam+tandegisken}{x=kaynakenlem+1} & 1 < tanx < 0\ and\ hedefboylam > kaynakboylam \\ \frac{y=kaynakboylam+tandegisken}{x=kaynakenlem+1} & -1 < tanx < 0\ and\ hedefboylam < kaynakboylam \\ \frac{y=kaynakboylam-tandegisken}{x=kaynakenlem-1} & -1 < tanx < 0\ and\ hedefboylam > kaynakboylam \\ \frac{y=kaynakboylam+1}{x=kaynakenlem+\frac{1}{tandegisken}} & 1 < tanx\ and\ hedefboylam > kaynakboylam \\ \frac{y=kaynakboylam-1}{x=kaynakenlem-\frac{1}{tandegisken}} & 1 < tanx\ and\ hedefboylam < kaynakboylam \\ \frac{y=kaynakboylam-1}{x=kaynakenlem-\frac{1}{tandegisken}} & -1 > tanx\ and\ hedefboylam < kaynakboylam \\ \frac{y=kaynakboylam+1}{x=kaynakenlem+1/tandegisken} & -1 > tanx\ and\ hedefboylam > kaynakboylam \end{cases} \quad (2)$$

def yasaklibolgekoseler(): global yasaklibolgekoselerarr boylambuyuk=yasaklibolgeler[0][0] boylamkucuk=yasaklibolgeler[3][0] enlembuyuk=yasaklibolgeler[0][1] enlemkucuk=yasaklibolgeler[3][1]	*Boylambuyuk yasaklibolgeler[0][0]* *boylamkucuk yasaklibolgeler[3][0]* *enlembuyuk yasaklibolgeler[0][1]* *enlemkucuk yasaklibolgeler[3][1]* değişkenleri atanır.

for i in range(len(yasaklibolgeler)):
if boylambuyuk< yasaklibolgeler[i][0]:
boylambuyuk=yasaklibolgeler[i][0]
if boylamkucuk >yasaklibolgeler[i][0]:
boylamkucuk=yasaklibolgeler[i][0]
if enlembuyuk< yasaklibolgeler[i][1]:
enlembuyuk=yasaklibolgeler[i][1]
if enlemkucuk> yasaklibolgeler[i][1]:
enlemkucuk=yasaklibolgeler[i][1]
	Yasaklibolgeler dizisindeki boylam en büyük, boylam en küçük, enlem en büyük, enlem en küçük değerlerinin tespiti

gecicilist=[] #sagust assagiya ve saga x1=yukari_kontrol(enlembuyuk,boylambuyuk) x2=saga_kontrol(enlembuyuk,boylambuyuk) x3=sola_kontrol(enlembuyuk,boylambuyuk) x4=asagi_kontrol(enlembuyuk,boylambuyuk)	Enlem en büyük ve boylam en büyük değerlerini *yukari_kontrol saga_kontrol sola_kontrol asagi_kontrol* fonksiyonları ile boylam en büyük, boylam en küçük, enlem en büyük, enlem en küçük değerlerinin hesaplanması
gecicilist.append(x2) gecicilist.append(x1) #solust yukariya ve sola	Geçici listeye *enlembuyuk* boylambuyuk değerlerinin eklenmesi

def saga_kontrol(enlem,boylam):
 	
bolge=cezeri.harita.bolge(enlem,boylam)
gecicidegisken=boylam
while (bolge.ucusa_yasakli_bolge):
global bolge
gecicidegisken=gecicidegisken+0.5
	Boylam değerini referans alarak sağa doğru bölge yasak mı diyerek kontrol eden ve etmeyen bölgeyi döndüren fonksiyon

bolge = cezeri.harita.bolge(enlem,gecicidegisken) return gecicidegisken +1	

def sola_kontrol(enlem,boylam):
 	
bolge=cezeri.harita.bolge(enlem,boylam)
gecicidegisken=boylam
while (bolge.ucusa_yasakli_bolge):
gecicidegisken=gecicidegisken-0.5
bolge = cezeri.harita.bolge(enlem,gecicidegisken)
return gecicidegisken-1
	Parametre olarak alınan enlem ve boylam değerlerini boylam 0.5 azaltılarak bölgeye atanmasıyla yasaklı bölge kontrol yaptıran döngü ve değerinin bir eksiği döndürmesidir.

def yukari_kontrol(enlem,boylam):
 	
bolge=cezeri.harita.bolge(enlem,boylam)
gecicidegisken=enlem
while (bolge.ucusa_yasakli_bolge):
gecicidegisken=gecicidegisken+0.5
bolge = cezeri.harita.bolge(gecicidegisken,boylam)
gecicienlem=gecicidegisken
return gecicidegisken+1
	Parametre olarak alınan enlem ve boylam değerlerini enlem 0.5 artırılarak bölgeye atanmasıyla yasaklı bölge kontrol yaptıran döngü ve değerinin bir fazlasını döndürmesidir. (Bir sonraki bölge değeri)

def asagi_kontrol(enlem,boylam):
 	
bolge=cezeri.harita.bolge(enlem,boylam)
gecicidegisken=enlem
while (bolge.ucusa_yasakli_bolge):
gecicidegisken=gecicidegisken-0.5
bolge = cezeri.harita.bolge(gecicidegisken,boylam)
return gecicidegisken-1
	Parametre olarak alınan enlem ve boylam değerlerini enlem 0.5 azaltılarak bölgeye atanmasıyla yasaklı bölge kontrol yaptıran döngü ve değerinin bir eksiği döndürmesidir (Bir sonraki bölge değeri)

4. Senaryo 7'de, trafikte olan diğer uçan arabaları gözeterek kaza yapmadan uçmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

I. Diğer hava araçlarının konumlarını hangi veri yapısından öğrendiniz?

II. Hava araçları ile kaza yapmamak için nasıl bir algoritma tasarladınız? Algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: Bulunduğumuz bölgede trafik olup olmadığı *cezeri.harita.bolge(kaynakenlem,kaynakboylam).trafik* değişkeninden alınan değere bakılarak anlaşılmıştır. Bu değişken bolean türündedir ve trafik varsa true değeri döndürmektedir. Hem bulunduğumuz konumda trafik var mı hem de bir sonraki bölgede (tanjant kadar ilerlettiğimiz bölgede) trafik var mı tespitini ve kontrolünü yaptırıp cezeri.dur*()* ile trafik geçinceye kadar aracın bekletilmesi sağlanmaktadır.

CevapII:

Algoritma	Açıklama

if(-1<tandegisken and tandegisken<1):
print('0<tandegisken and tandegisken<1')
if(tandegisken>0):
print('tandegisken>0')
 	
if(hedefboylam>kaynakboylam):
 	
kaynakboylam=tandegisken+kaynakboylam
 	
kaynakenlem=kaynakenlem+1
 	
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
if bolge.trafik or bolge2.trafik:
 	
print('bolge.trafik1',bolge.trafik)
while(bolge.trafik or bolge2.trafik):
	Bu senaryodada, senaryo 4'de geliştirilen algoritmanın benzeri uygulanmıştır. Aracın rotası küçük bölgelere ayrılarak (Şekil 2) her bölgede trafik kontrolü yapılmaktadır. Trafik olması durumunda aracın bekletilmesi, trafiğin geçmesi halinde ise tekrar hareket etmesi sağlanmaktadır. Bölgelerin ayrılması ve ayrılan bölgelerin enlem-boylam bilgileri Eşitlik 1 ve Eşitlik 2'de gösterildiği şekilde hesaplanmaktadır.

```
print('bolge.trafik2',bolge2.trafik)
                        cezeri.dur()
                        cezeri.bekle(0.5)
else:

kaynakboylam=kaynakboylam-
tandegisken

kaynakenlem=kaynakenlem-1

bolge=cezeri.harita.bolge(kaynakenlem,ka
ynakboylam)
                bolge2          =
cezeri.harita.bolge(cezeri.gnss.enlem,cezer
i.gnss.boylam)
                if      bolge.trafik    or
bolge2.trafik:

print('bolge.trafik1',bolge.trafik)
                    while(bolge.trafik    or
bolge2.trafik):

print('bolge.trafik2',bolge2.trafik)
                        cezeri.dur()
                        cezeri.bekle(0.5)
if(tandegisken<-1):
            print('tandegisken<-1')

if(kaynakboylam<hedefboylam):#zeceri
sol tarafta

kaynakboylam=kaynakboylam-
tandegisken
              kaynakenlem=kaynakenlem-1
```

```
bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
                bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
                if bolge.trafik or bolge2.trafik:

print('bolge.trafik1',bolge.trafik)
                    while(bolge.trafik or bolge2.trafik):

print('bolge.trafik2',bolge2.trafik)
                        cezeri.dur()
                        cezeri.bekle(0.5)
kaynakboylam=kaynakboylam+tandegiskeen

kaynakenlem=kaynakenlem+1

bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
                    bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
                    if bolge.trafik or bolge2.trafik:

print('bolge.trafik1',bolge.trafik)
                        while(bolge.trafik or bolge2.trafik):

print('bolge.trafik2',bolge2.trafik
                            cezeri.dur()
                            cezeri.bekle(0.5)
```

```
print('hedefboylam4',hedefboylam,'hedefenlem4',hedefenlem)

print('kaynakboylam4',kaynakboylam,'kaynakenlem4',kaynakenlem)

bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
if(tandegisken>1):
        if(cezeri.manyetometre.veri>0):
            print('tandegisken>1')

kaynakboylam=kaynakboylam+1/2

kaynakenlem=(1/tandegisken*2)+kaynakenlem

bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
            bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
            if bolge.trafik or bolge2.trafik:

print('bolge.trafik1',bolge.trafik)
                while(bolge.trafik or bolge2.trafik):

print('bolge.trafik2',bolge2.trafik)
                    cezeri.dur()
                    cezeri.bekle(0.5)
else:
            print('tandegisken>0')
        #while(abs(kaynakboylam-cezerileyasakkoseler[eleman,0])>5):
```

```
kaynakboylam=kaynakboylam-1/2
                kaynakenlem=kaynakenlem-
(1/tandegisken*2)

bolge=cezeri.harita.bolge(kaynakenlem,ka
ynakboylam)
                bolge2                =
cezeri.harita.bolge(cezeri.gnss.enlem,cezer
i.gnss.boylam)

print('tandegisken',tandegisken)
                if bolge.trafik or bolge2.trafik:

print('bolge.trafik1',bolge.trafik)
                    while(bolge.trafik          or
bolge2.trafik):

print('bolge.trafik2',bolge.trafik)
                        cezeri.dur()
                        cezeri.bekle(0.5)

print('bolge.trafik3',bolge.trafik)

print('hedefboylam3',hedefboylam,'hedefen
lem3',hedefenlem)
elif(tandegisken<-1):
          print('tandegisken<-1')
          if(cezeri.manyetometre.veri>0):

kaynakboylam=kaynakboylam+(1/(tandegi
sken*2))

kaynakenlem=kaynakenlem+1/2
```

```
print('hedefboylam1',hedefboylam,'hedefenlem1',hedefenlem)

print('kaynakboylam1',kaynakboylam,'kaynakenlem1',kaynakenlem)

bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
            bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
            if bolge.trafik or bolge2.trafik:
                while(bolge.trafik or bolge2.trafik):
                    cezeri.dur()
                    cezeri.bekle(0.5)
else:

kaynakboylam=kaynakboylam-1/2
            kaynakenlem=kaynakenlem-(1/tandegisken*2)

print('hedefboylam2',hedefboylam,'hedefenlem2',hedefenlem)

print('kaynakboylam2',kaynakboylam,'kaynakenlem2',kaynakenlem)

bolge=cezeri.harita.bolge(kaynakenlem,kaynakboylam)
            bolge2 = cezeri.harita.bolge(cezeri.gnss.enlem,cezeri.gnss.boylam)
            if bolge.trafik or bolge2.trafik:
```

print('bolge.trafik2',bolge.trafik) while(bolge.trafik or bolge2.trafik): cezeri.dur() print('bolge.trafik2',bolge.trafik) cezeri.bekle(0.5)	

5. Senaryo 8'de, en yakın hastaneyi bulmanızı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

I. Hastanelerin olduğu veri yapısı nedir?

II. En yakın hastanenin seçimini yaptığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız

CevapI: *cezeri.acil_durum* değişkeni true değerini aldığında aracımızın o anki konumuna en yakın hastaneyi seçmek için *cezeri.harita.hastaneler* listesini tarayıp en kısa mesafedeki hastaneye gidilmektedir.

CevapII:

Algoritma	Açıklama

def en_yakin_hastane():
if cezeri.acil_durum:
global varis_bolgesi
gecicidegisken=100000
for i in range(len(cezeri.harita.hastaneler)):
mesafe2=mesafe(cezeri.gnss.boylam,cezeri.gnss.enlem,cezeri.harita.hastaneler[i].boylam,cezeri.harita.hastaneler[i].enlem)
if(mesafe2<gecicidegisken):
gecicidegisken=mesafe2
hastanenumarasi=i
varis_bolgesi=cezeri.harita.hastaneler[hastanenumarasi]
	cezeri.acil_durum değişkeni true değerini aldığında, *cezeri.harita.hastaneler* listesi taranmaktadır. Listede yer alan hastanelerin enlem ve boylam verileri kullanılarak öklid yöntemiyle aracımızla aralarındaki mesafeler hesaplanmıştır. Buna göre en kısa mesafedeki hastane tespit edilmiştir.

6. Senaryo 10 içerisinde GNSS arızası yaşandığında inişi tamamlayan algoritma için aşağıdaki soruları cevaplayınız.

I. GNSS arızasını hangi veri yapısından tespit ettiniz?

II. Bu arıza sonrası hava aracının irtifasını hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: *cezeri.gnss.hata* değişkeni kullanırak tespit edilmektedir. Değişken true değerini alırsa hata olduğu anlaşılmaktadır.

CevapII:

Algoritma	Açıklama
cezeri.barometre.yukseklik	Arıza durumunda özel bir algoritmaya gerek olmaksızın *cezeri.barometre.yukseklik* verisinden faydalanılarak irtifa bilgisine ulaşılmaktadır.

7. Senaryo 11 içerisinde GNSS arızası yaşandığında seyir yapmaya devam eden ve görevi tamamlamayı sağlayan algoritma için aşağıdaki soruları cevaplayınız.

I. Bu arıza sonrası hava aracının pozisyonunu(yatay) hesapladığınız algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI:

Algoritma	Açıklama
if cezeri.gnss.hata: mesafedegiskeni=mesafe(soncezeriboylam, soncezerienlem,varis_bolgesi.boylam,varis_bolgesi.enlem) hizvektoru=math.sqrt(pow(cezeri.imu.Vx,2)+pow((cezeri.imu.Vy),2)) sure=mesafedegiskeni/hizvektoru cezeri.ileri_git(cezeri.hizli) cezeri.bekle(sure) cezeri.dur() hedefevarildimi=True varilanhedefsayisi=varilanhedefsayisi+1	GNSS sensöründe hata oluşması durumunda, hatanın oluştuğu anda aracın konumu ile hedef arasındaki mesafenin hesaplanması için *"mesafe"* fonksiyonu çağırılır. Mesafe öklit yöntemiyle hesaplanmaktadır. Başlangıç anındaki hedefe olan mesafeden arıza anındaki mesafe çıkarılarak güncel mesafe bilgisine ulaşılamaktadır. Daha sonra, aracın hızı (V) için Eşitlik 3'de verilen şekilde hız vektörü hesaplanır. $V = \sqrt[2]{cezeri.imu.Vx^2 + cezeri.imu.Vy^2}$ (3) Buradan, $süre = mesafe/V$ hesabıyla aracın hedefe varmak için gitmesi gereken süre hesaplanır. Bu süre kadar araç ilerletilip inmesi sağlanır.

8. Senaryo 12'de GNSS arıza durumunu tespit eden algoritma için aşağıdaki soruları cevaplayınız.

I. GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir?

II. GNSS yanıltma (spoofing) durumunu tespit ettiğiniz algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: Spoofing, GNSS alıcısının yanlış konum hesaplaması anlamına gelirken jamming, alıcının artık çalışamaması için yerel olarak GNSS uydu sinyallerini aşırı güçlendirmesidir.

CevapII:

Algoritma	Açıklama

def spoofing_varmi():
global baslangictan
global spoofingbool
if (2==len(cezerikonumlist)):
cezerikonumlist.append([float(cezeri.gnss.enlem),float(cezeri.gnss.boylam)])
cezerikonumlist.pop(0)
if(abs(cezerikonumlist[0][1]-cezerikonumlist[1][1])>1 and abs(cezerikonumlist[0][0]-cezerikonumlist[1][0])>1):
cezeri.dur()
spoofingbool=True
else:
cezerikonumlist.append([cezeri.gnss.enlem,cezeri.gnss.boylam])
	While dögüsünün içinde *spoofing_varmi* fonksiyonu çağırılarak karıştırma kontrolü yapılmaktadır. Fonksiyon içinde, aracın konumları sürekli olarak *cezerikonumlist* ismiyle tanımlanan diziye kayıt edilmektedir. Dizi kaydedilen son iki konum arasındaki farkın mutlak değeri alınarak, eğer 1'den büyükse karıştırmaya (spoofing) maruz kaldığımız anlaşılmaktadır.

9. Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimini yaptığınız algoritma için aşağıdaki soruları cevaplayınız.

I. İniş için hangi sensörü irtifa kaynağı olarak seçtiniz? Neden?

II. Diğer sensörleri neden seçmediniz?

III. Bu seçimi yapan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: Lidar sensörü kullanılmıştır. Senaryo gereği, barometre, radar ve gnss sensörlerinde veri kaybı yaşandığından irtifa kaynağı olarak sadece lidar kullanılabilmektedir.

CevapII: Senaryo gereği, barometre, radar ve gnss sensörleri very kaybı yaşandığından tercih edilmemiştir.

CevapIII:

Algoritma	Açıklama

if(cezeri.gnss.hata or cezeri.radar.hata or cezeri.barometre.hata):
if(cezeri.lidar.hata!=True):
while cezeri.lidar.mesafe >16
cezeri.asagi_git(cezeri.hizli)
 	
cezeri.asagi_git(cezeri.yavas)
else
 	
cezeri.asagi_git(cezeri.yavas)
	İniş esnasında lidar dışında irtifa bilgisi veren diğer sensörlerde veri kaybı yaşanması durumunda iniş lidar sensörüyle yapılmaktadır. Ancak, iniş esnasında lidar sensörde de veri kaybı yaşanması durumunda, aracın sensörsüz olarak yavaş hızda yere inmesi sağlanmaktadır.

10. Senaryo 15'te manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe hava aracının yönelmesini sağlayan algoritma için aşağıdaki soruları cevaplayınız.

I. Manyetometre veri kaybını nasıl tespit ettiniz?

II. Hava aracının gitmesi gereken yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

III. Hava aracının gittiği yönü hesaplayan algoritmayı EK-1'de verilen örnekteki gibi açıklayınız.

CevapI: c*ezeri.manyotemetre.hata* değişikeni true değeri almışsa ve muhtemel gecikme yaşanması durumunda *cezeri.manyetometre* değeride control edilerek değerin 0 olması durumunda manyometrede veri kaybı yaşandığı anlaşılmaktadır.

CevapII:

Algoritma	Açıklama

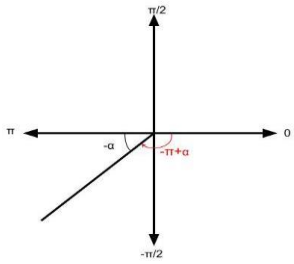
def manyetometre():
global cezerikonumlist
global manyetometreaci
print('manyetometre')
if (2==len(cezerikonumlist)):
cezerikonumlist.append([float(cezeri.gnss.enlem),float(cezeri.gnss.boylam)])
cezerikonumlist.pop(1)
	Bu senaryo için, algoritmamızda manuel bir manyetometre geliştirilmiştir. Aracın konumu (enlem-boylam) sürekli alınarak *cezerikonumlist* dizisine aktarılır. Dizi içindeki son iki konum arasındaki açı *tan1* fonksiyonu ile hesaplanır ve sonuç *tandegisken* değerine aktarılır. Bu değerin aldığı değere göre aracın döneceği açıyla

```
print(cezerikonumlist,'cezerikonumlist')

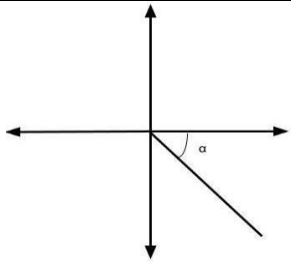
tandegisken=tan1(cezerikonumlist[0][1],cezerikonumlist[0][0],cezerikonumlist[1][1],cezerikonumlist[1][0])
      if(tandegisken is not None):
         boylamfark=cezerikonumlist[1][1]-cezerikonumlist[0][1]#boylamfark
         enlemfark=cezerikonumlist[1][0]-cezerikonumlist[0][0]#enlemfark
         aci=math.atan(tandegisken)
         print('tanmanyetometreaci',aci)
if(boylamfark<0 and enlemfark<0):#3.bolge
            cezerikonumlist.pop(0)
            manyetometreaci= -math.pi+aci
         if(boylamfark>0 and enlemfark<0):#2.bolge
            cezerikonumlist.pop(0)
            manyetometreaci= math.pi+aci
         if(boylamfark<0 and enlemfark>0):#4.bolge
            cezerikonumlist.pop(0)
            manyetometreaci= aci
         if(boylamfark>0 and enlemfark>0):#1.bolge
            cezerikonumlist.pop(0)
            manyetometreaci= aci
      else:
print(cezerikonumlist,'cezerikonumlist')
         return None
   else:
cezerikonumlist.append([cezeri.gnss.enlem,cezeri.gnss.boylam]
```

ilgili gösterimler surasıyla Şekil 3, Şekil 4, Şekil 5 ve Şekil 6'da verilmiştir.

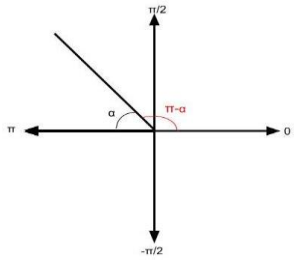
Şekillerden de anlaşılacağı üzere, α açısı, son iki nokta arasındaki açıyı temsil eder. Bu açı değerine göre aracın dönmesi gereken yönü ve dönüş açısı radian cinsinden hesaplanır.



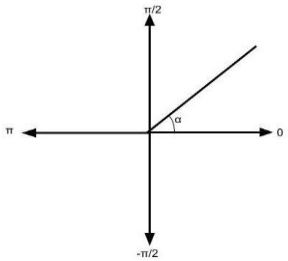
Şekil 3. bölge 3 deki aciyi temsil eden manyetometre verisi



Şekil 4. 1bölge 4deki aciyi temsil eden manyetometre verisi



Şekil 5. 1bölge 2 deki aciyi temsil eden manyetometre verisi



Şekil 6. 1bölge 2 deki aciyi temsil eden manyetometre verisi

CevapIII:

Algoritma	Açıklama
boylamfark=cezeri.gnss.boylam-varis_bolgesi.boylam enlemfark=cezeri.gnss.enlem-varis_bolgesi.enlem tan=boylamfark/enlemfark aci=math.atan(tan)	Boylam fark ve enlem fark değerlerimizden sonra tanjantı hesapladıktan sonra arktanjant fonksiyonu ile açı değerini radyan cinsinden hesaplayan kod bloğudur.

if(manyetometreaci is not None):
print('manyetometreaci2',manyetometreaci)
print('tan',tan)
if(aci-manyetometreaci>0):
if(cezeri.gnss.enlem<varis_bolgesi.enlem):
#1.bolgecezeri.don(aci-manyetometreaci)
cezeri.bekle(0.1)
print('a2')
else:
cezeri.don((aci-manyetometreaci-math.pi))#3.bolge
	Açı değerimiz manyetometre değerinden büyükse 1.bölge ve 3. bölgededir.
	Eğer ki cezeri enlemimiz varış bölgesi enlemimizden daha büyükse 1. bölgededir ve açı kadar döndürülmelidir. Değilse 3. bölgededir ve dönüşü yapılır.

cezeri.bekle(0.1) print('b2')	

else:
if(cezeri.gnss.enlem<varis_bolgesi.enlem):
#2.bolge
cezeri.don(aci-manyetometreaci)
cezeri.bekle(0.1)
print('c2')
else:
cezeri.don((aci-manyetometreaci+math.pi))#4.bolge
cezeri.bekle(0.1)
print('d2')
	Açı değerimiz manyetometremizden büyük değilse ve cezeri enlem varış bölgesinden büyükse 2. bölgededir ve buna göre dönmelidir.
	Değil ise 4.bölgededir ve buna göre dönmelidir.

11. Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarını tespit ettiğiniz ve hava aracını oraya yönlendirdiğiniz algoritma için aşağıdaki soruları cevaplayınız.

I. Batarya durumlarını hangi veri yapısından öğrendiniz?

II. Batarya durumunun hangi gerilim eşiğinde şarj istasyonuna inilmesi gerektiğini seçtiniz? Neden?

III. Şarj istasyonlarının olduğu veri yapısı nedir?

CevapI: *cezeri.batarya.veri,* Uçan arabanın batarya doluluk bilgisini verir. Float türünde değer döndürür.

CevapII: (*cezeri.batarya.veri*-20)<(*mesafelist[1]***ortalamabataryatuketimi*) Yukarıdaki kod satırında ifade edilen değişkenler aşağıda açıklanmıştır;

Cezeri.batarya.veri: Uçan arabanın batarya doluluk bilgisidir.

Mesafelist []: Hedefler arasındaki mesafelerin tutulduğu dizidir.

Ortalamabataryatuketimi: Aracın iki nokta arasındaki mesafeyi tutan değişkendir. Şu şekilde hesaplanır; *bataryatüketimi/katedilenmesafe*

CevapIII: *cezeri.harita.sarj_istasyonlari,* Harita üzerinde bulunan bütün sarj istasyonlarının saklandığı dizidir. Bu senaryoda cezerinin mevcut enlem ve boylam değerleri ile dizideki şarj istasyonlarının enlem ve boylam değerleri üzerinden mesafeler hesaplanır ve en kısa mesafedeki şarj istasyonuna araç yönlendirilir.

1. Kaynakça

1-https://bilginc.com/tr/blog/307/python-ogrenmenin-avantajlari

2-https://makinecim.com/bilgi_7409_sensorler-

.../.../2022

TEKNOFEST YARIŞMALAR KOMİTESİ'NE

Yukarıda belirttiğim takıma ait yarışma sürecinde oluşabilecek tüm sorumluluk danışman olarak bana ait olduğunu bildiririm.

AD SOYAD	Naim KARAKEKETER
T.C KİMLİK NO	12310451366
CEP TELEFONU	05059332557
KURUM	Afyon Kocatepe Üniversitesi Müh. Fak. Bilgisayar Müh. Bölümü
GÖREVİ	Dr. Öğr. Üyesi
YARIŞMA ADI / KATEGORİ	Uçan Araba Yarışması/Üniversite ve Üzeri
DANIŞMAN OLDUĞU TAKIM ADI	AKÜ_RISECAR

DANIŞMAN

AD SOYAD

İMZA

Naim KARAKEKETER

.../.../2022

TEKNOFEST YARIŞMALAR KOMİTESİ'NE

AD SOYAD	Naim KARASEKRETER
T.C KİMLİK NO	12310451366
CEP TELEFONU	05059332557
KURUM	Afyon Kocatepe Üniversitesi Müh. Fak. Bilgisayar Müh Bölümü
GÖREVİ	Dr. Öğr. Üyesi

Yukarıda bilgileri belirtilen personelin kurumumuzda öğretmen/eğitmen/akademisyen olarak görev aldığını bildiririm.

KURUM
(İMZA/KAŞE)

T.C
Şükrü Gönen
Fakültesi

PERSONEL
AD SOYAD
İMZA

Naim KARASEKRETER